Choose Your Own Project: Handwritten Character Recognition

edX HarvardX: PH125.9x - Data Science: Capstone

Azamat Kurbanaev

2026-10-19

Contents

Preface **3**

Acknowledgements **4**

1 Overview and Executive Summary **5**

1.1 Project Description 5

1.2 Environment Setup 7

1.2.1 Batch Execution & Cleanup Scripts 7

1.3 Running the Project 8

1.3.1 The Main `R` Script 9

1.3.1.1 Setup & Initialization 9

1.3.1.2 Downloading & Preprocessing Input Datasets . . 11

1.3.1.3 Machine Learning Model Pipelines 11

1.3.1.4 Logging & Utility Features 12

1.4 Datasets Overview 12

2 Methods and Analysis **13**

2.1 k-Nearest Neighbors + Principal Component Analysis (kNN+PCA) Multiclass Classifier (MCC) Model 13

2.2 Random Forest (RF) MCC Model 13

2.3 Dense Neural Network-Based Basic Multiclass Classifier (DNNB MCC) Model 13

2.4 Tuned DNN-Based (TDNN) MCC Model 13

2.5 Convolutional Neural Network-Based Basic (CNNB) MCC Model 13

2.6 Tuned CNN-Based (TCNN) MCC Model 13

3 Results **14**

4 Conclusion **15**

5 Appendix A: Support Functions **16**

5.1 Data Helper Functions 16

5.1.1 Data Loading 16

5.1.1.1 load_image.list Function 16

5.1.1.2 img.file_path.get_list Function 20

5.1.1.3 load.img_trimmed.dat Function 23

5.1.2 Image Processing {#appndx_a@img_proc} 25

5.1.2.1 magick_img2matrix Function 25

6 ————————- **27**

7 ————————- **28**

Preface

This project is the second assignment of the 'Data Science: Capstone' course (PH125.9x), offered by edX HarvardX. The project aims to build and compare different Handwritten Character Recognition (HCR) models, applying different Machine Learning (ML) techniques, and using a publicly available dataset for training and validation.

Acknowledgements

I would like to extend my deepest gratitude to Professor **Rafael A. Irizarry** for designing and delivering the comprehensive courses within the ***Professional Certificate Program in Data Science***. Prof. **Irizarry**'s instruction has provided me with the essential theoretical foundations and practical toolkit required for modern data analysis.

I am equally grateful for his textbook, *Introduction to Data Science*, (part 1[1] and part 2[2]). This text served as an indispensable guide throughout my coursework, offering clear explanations and robust examples that bridged the gap between theory and execution. Prof. **Irizarry**'s dedication to creating clear, accessible educational material has profoundly enhanced my learning experience and will undoubtedly continue to serve as a key reference in my future endeavors.

Chapter 1

Overview and Executive Summary

As noted in the article Handwritten Character Recognition: A Comprehensive Survey, Handwritten Character Recognition (HCR) plays a vital role in numerous domains, including document digitization, automatic text recognition, signature verification, and postal automation. Over the years, HCR has advanced significantly, driven by the growing demand for accurate, efficient recognition systems [3].

1.1 Project Description



All the source files for *this Project* are available in the *Project*'s GitHub repository[4].

In this project, we will develop and explore several HCR models, based on popular ML algorithms that convert handwritten alphanumeric symbols into a digital, machine-readable form.

We will start with Shallow Learning (SL) models, using k-Nearest Neighbors (kNN) and Random Forest (RF) methods. These models development and evaluation are described in detail below in Sections k-Nearest Neighbors + Principal Component Analysis (kNN+PCA) Multiclass Classifier (MCC) Model and Random Forest (RF) MCC Model of *this Report*, respectively.

Next, we will try Deep Learning (DL) models, including Dense Neural Network (DNN) and, finally, Convolutional Neural Network (CNN) models, as recent re-

search shows that DL performs better at image recognition — the task addressed by *this Project*.

A more detailed explanation of *Deep Learning* versus *Shallow Learning* can be found in the chapter Machine Learning Methods from Shallow Learning to Deep Learning of the book Shallow Learning vs. Deep Learning: A Practical Guide for Machine Learning Solutions by editors Ömer Faruk Ertuğrul, Josep M. Guerrero, and Musa Yilmaz[5].

The ***DNN-Based*** models are as follows:

- ***DNN-Based Basic MCC Model:*** described in Section Dense Neural Network-Based Basic Multiclass Classifier (DNNB MCC) Model;
- ***Tuned DNN-Based MCC Model:*** described in Section Tuned DNN-Based (TDNN) MCC Model.

The last models we will build and explore, the ***CNN-Based*** models, are as follows:

- ***CNN-Based Basic MCC Model:*** described in Section Convolutional Neural Network-Based Basic (CNNB) MCC Model;
- ***Tuned CNN-Based MCC Model:*** described in Section Tuned CNN-Based (TCNN) MCC Model.

Finally, we will conduct a final test of the best model we have developed, using a specially designated dataset, and see what we will achieve.

1.2 Environment Setup



The environment setup instructions are provided in the Prerequisites & Setup section of the README document on GitHub.

The environment setup configures an `R`-based machine learning environment with *Python* integration, required to build the *Deep Learning* models.

The installation follows a *4-step* process by sequentially running the following scripts:

- 1.install-reticulate&miniconda.R:
  1. *Installs reticulate:* Configures the primary `R` package, the R Interface to Python;
  2. *Installs Miniconda* to manage Python virtual environments cleanly without conflicting with system Python installations.

- 2.install-tensorflow.R:
  1. **Installs the R TensorFlow interface:** Loads the TensorFlow R bindings;
  2. **Configures Backend Engine:** Uses reticulate to set up a dedicated *Python virtual/conda environment* and installs the core C++/Python TensorFlow package inside it.

- 3.install-keras3.R:
  1. **Installs the Keras3 package:** Installs the R Interface for Keras3.
  2. **Binds High-Level Neural Network APIs:** Integrates `Keras` on top of the backend installed in step 2 to enable building and training *Deep Learning* models in `R`.

- 4,install-kerastuner.R: Installs ***Hyperparameter Tuning Tools***, including R interface to Keras Tuner. Configures the KerasTuner in `R` to automate Hyperparameter Optimization (*search space*, *trial execution*, and *model evaluation*).

1.2.1 Batch Execution & Cleanup Scripts

As stated in the Prerequisites & Setup section of the README document on GitHub, on *Windows OS*, all 4 steps of the setup process described above can be completed automatically by launching the ___setup-env.cmd *Windows command script* in *PowerShell* or *Command Prompt*.

The script ___uninstall-all.R cleans up the environment by wiping installed packages, *Conda environments*, and *Python bindings*.

1.3 Running the Project

Once the system environment is fully configured, if you have enough time to execute all the scripts and automatically generate *this report* upon their completion in a single session, the easiest approach is to run the ___run-all.R script.

This script serves as a starting point for running the project execution pipeline that culminates in a *PDF report*. It coordinates the process by sequentially executing two main R scripts, ensuring the script files exist, and outputs detailed execution logs to both the console and log files.

The first script it launches, index.R, is the entry point to the project's execution pipeline and the main script that runs all the other execution scripts sequentially.

The second (and last) script it runs, ___rmd.render.R, renders *this Report*.



Some scripts may take a long time when executed on their first run. In particular, the longest run on the author's laptop was the first run of the 2.cnn-mcc.lr-tuner.R script (for fine-tuning the *CNN-based MCC* model) and took about **17 hours**.

The other long-running scripts are as follows:

- For tuning and retraining the best (and *final*) *CNN-based MCC* model:
  - 1.cnn-mcc.arch-tuner.R (took more than **6 hours** on the author's laptop);
  - 3.tcnn-mcc.final.retrain.R (took around **6.5 hours**);
- For tuning, retraining, and evaluating the best *kNN+PCA MCC* model:
  - 3.knn+pca.best-eval.R (took more than **2 hours**);
  - 2.knn+pca.re-train.best-k.R (took about **2 hours**);
  - 1.knn+pca.build&tune.R (about *half an hour*).
- For tuning and retraining the best *Random Forest (RF) MCC* model:
  - 2.rf-retraining.best-par.R (more than **1.5 hours**).
  - 1.rf-tuning.R (more than ***an hour***).
- For tuning the *DNN-Based MCC* model:
  - 1.dnn-mcc.tuner.R (more than *half an hour*).

- For downloading, analyzing, and preprocessing datasets:
  - ds.kaggle.download.R (about ***an hour***).
  - ds.find-out.img.min.size.R (about *half an hour*).
  - prepare-input-data.R (more than *half an hour*).
  - ds.train2trimmed_img.dat.R (about ***an hour***).



Subsequent runs of these long-running scripts will be much faster thanks to a caching mechanism that checks for the existence of the pre-computed results before running time-consuming training or evaluation code and loads the results from the cache files (if any). Besides, any of these scripts can be launched separately, or the execution can be interrupted at any time and resumed later from the point of interruption.



If the interrupted execution resumes in another *`R` session*, the Setup section (with all its subsections) of the index.R script **must be run first at the very beginning of the session**.

1.3.1 The Main `R` Script

The index.R script serves as the main orchestrator and *entry point* for *this Project*, focused on Multi-Class Classification (MCC) models applied to the Kaggle Handwritten Character Image Dataset (which uses EMNIST data for Alphabets and Digits). Below is a detailed description of the script's structure, workflow, and core components.

1.3.1.1 Setup & Initialization

The Setup section of the index.R code configures and initializes the *Global Environment* for the current `R` session, along with downloading, preparing, and preprocessing the project's datasets.



Code in this section must run at the very beginning of every *`R` session* **before any other script executes**.

The code in this section first calls the r-global.path.R script (on line 6) to initialize global variables for the following `R` scripts' directory paths:

- root `R` scripts' directory;

- support scripts' directory;
- user-defined support (helper) functions' scripts directory.

Next (on line 10), it executes the setup.R script, which installs the full set of required packages (those not yet installed), loads the required libraries, and completes the remaining initialization.

Next, in the Download the Kaggle Dataset subsection, it downloads the Kaggle dataset, executing the ds.kaggle.download.R script.

The last subsection, Prepare Input Data, prepares the project's datasets. It is, in turn, divided into the following subsections:

- Train Set: Create Trimmed Image Data List:
- :

1.3.1.1.1 setup.R Script

This script serves as the master initialization and environment-configuration script for the entire project. Its primary purpose is to set up and configure all software dependencies, Python environments, directory paths, and helper functions before running any project execution scripts.

The first two lines of the code perform the following operations in order:

1. Adjust the global download timeout to handle packages and large dataset downloads;
2. Increase the expression nesting limit.

Below is a detailed breakdown of the code by sections.

1.3.1.1.1.1 Install Packages Section

The code in this section automatically checks for required `R` packages across multiple domains and installs them if some or all are not installed.

1.3.1.1.1.2 Load Libraries & resolve conflicts Section

1.3.1.1.1.3 Load Logging Helper Functions Section

1.3.1.1.1.4 Init Project Directories Section

1.3.1.1.1.5 Init Input Data Paths Section

1.3.1.1.1.6 Init Project Script Paths Section

1.3.1.1.1.7 Load Common Helper Functions Section

1.3.1.1.1.8 Load Data Helper Functions Section

1.3.1.1.1.9 Load Model Helper Functions Section

1.3.1.1.1.10 Load `CNN_MCC.HyperModel` helper class Section

1.3.1.1.1.11 Load `CNN_MCC.HyperParameters` helper class Section

1.3.1.2 Downloading & Preprocessing Input Datasets

1.3.1.2.1 prepare-input-data.R Script

1.3.1.3 Machine Learning Model Pipelines

1.3.1.3.1 k-Nearest Neighbors with Principal Component Analysis (kNN + PCA) Model

1.3.1.3.2 Random Forest (RF) MCC Model

1.3.1.3.3 Deep Learning Models

1.3.1.3.3.1 Models Based on Dense Neural Network (DNN)

####### Basic DNN-Based Model

Tuned DNN-Based Model

1.3.1.3.3.2 Models Based on Dense Neural Network (DNN)

Basic CNN-Based Model

Tuned CNN-Based Model

1.3.1.4 Logging & Utility Features

1.4 Datasets Overview

The project uses the HandWritten_Character Dataset downloaded from the Kaggle platform.

As described on the landing web-page, the *context* of the dataset is as follows:

1. Used *EMNIST* data for Alphabets and Digits.
2. Transformed the Data using some image processing techniques and convert it to 32,32 pixel black and white images.
3. Created the Data set for special character (@, #, $, &)
4. Merged the Categories to avoid the misclassification
5. Total 39 Categories in Train and Validation set

---

using a Kaggle Dataset.

Chapter 2

Methods and Analysis

2.1 k-Nearest Neighbors + Principal Component Analysis (kNN+PCA) Multiclass Classifier (MCC) Model

2.2 Random Forest (RF) MCC Model

2.3 Dense Neural Network-Based Basic Multiclass Classifier (DNNB MCC) Model

2.4 Tuned DNN-Based (TDNN) MCC Model

2.5 Convolutional Neural Network-Based Basic (CNNB) MCC Model

2.6 Tuned CNN-Based (TCNN) MCC Model

Chapter 3

Results

Chapter 4

Conclusion

Chapter 5

Appendix A: Support Functions

5.1 Data Helper Functions

5.1.1 Data Loading



The complete source code of the functions described in this section is available in the the Data loading section of the data-helper.R script on *GitHub*.

5.1.1.1 load_image.list Function

This is a core utility designed to recursively traverse a root directory passed in the root_path argument, discover character-specific (or class-specific, in the terms of the project's multiclass classifier models) subfolders, load data from raw image files (downloaded from the Kaggle platform) using a custom parsing function, the pointer to which is passed in the fn.load_image.dat argument, filter out empty or invalid images, and organize everything into an `R` list object with the corresponding labels and file paths.

5.1.1.1.1 Usage

```
train.img_dat.list <- load_image.list(root_path = img.train_root.dir,
                                      fn.load_image.dat = load.img_trimmed.dat,
                                      folder.list = NULL,
                                      char_files.max = NA,
                                      char_files.seed = NA,
                                      random_sample = FALSE)
```



In the *code snippet* above, a pointer to the auxiliary function load.img_trimmed.dat described below in *this Section* is used as the value of the argument fn.load_image.dat.

5.1.1.1.2 Arguments

- **root_path:** The root directory path containing subfolders of images organized by character recognition classes;
- **fn.load_image.dat:** Pointer to a user-defined loading and processing helper function applied to each image file path (e.g., a function to read image matrices or binary arrays);
- **folder.list:** (Optional, *Character*, $NULL$ by default) Vector of specific subfolder names to process. If $NULL$, it defaults to all directories found inside the *root path* passed in the root_path argument;
- **char_files.max:** (Optional, *Integer*, NA by default) Numeric value limiting the maximum number of files to load per folder (useful for subsampling or testing);
- **char_files.seed:** (Optional, *Integer*, NA by default) Random seed used for reproducible sampling when char_files.max is specified (ignored otherwise);
- **random_sample:** (Optional, *Boolean*, $FALSE$ by default) Boolean ($TRUE/FALSE$) indicating whether files within a folder should be sampled randomly ($TRUE$) or taken sequentially from the top ($FALSE$).

5.1.1.1.3 Details

Below is the *step-by-step execution flow* of the function:

1. **File Path Discovery:** Internally invokes the img.file_path.get_list user-defined helper function (described below in the Image Processing section) to gather a structured list of file paths from each target subfolder, applying optional maximum limits (passed in the char_files.max argument), random sampling (depending on the value passed in the random_sample argument), and seed generation (if the char_files.seed argument value is not NA).
2. **Label Generation:** Extracts directory names from the file path list and converts them into a factor to serve as classification labels.
3. **Iterative processing of class directories:** Iterates through each directory, performing the following operations:
    a) (Lines 109-110) **Batch Loading:** Calls the custom loader function (the pointer to which is passed in the fn.load_image.dat argument) using the map `R` function across all file paths in the folder.
    b) (Lines 112-116, 119-120) **Data Validation & Filtering:** Evaluates a validity check on the loaded image data. Specifically, it checks that the binary matrix elements sum to greater than zero, effectively filtering out blank, corrupt, or unreadable images.
4. (Line 122) **Consistency Checks:** Enforces a safety assertion to ensure that the filtered image list length strictly matches the corresponding filtered file path list length.

5.1.1.1.4 Value

Structured list containing two primary components: 1. *Label list:* A factor of *class labels*; 2. *Image list:* A nested list mapping each *class folder* to its valid loaded *image data* and corresponding *file paths*.

5.1.1.1.5 Source Code

The simplified version of the source code function is shown below:

```
load_image.list <- function(root_path,
                            fn.load_image.dat,
                            folder.list = NULL,
                            char_files.max = NA,
                            char_files.seed = NA,
                            random_sample = FALSE) {

  img.file_list <- img.file_path.get_list(root_path,
                                          folder.list,
                                          char_files.max,
                                          char_files.seed,
                                          random_sample)

  label_list <- as.factor(names(img.file_list))
  img_list <- lapply(img.file_list, function(char.dir){

    img_dat.ls <- map(char.dir$file_path.list,
                  fn.load_image.dat)

    valid.idx <- sapply(img_dat.ls, function(img.dat){
      mx.bin <- img.dat[1,,]
      mode(mx.bin) <- "numeric"

      sum(mx.bin) > 0
    })

    ls <- list(img.list = img_dat.ls[valid.idx],
               fpath.list = char.dir$file_path.list[valid.idx])

    stopifnot(length(ls$img.list) == length(ls$fpath.list))

    ls
  })

  list(label.list = label_list,
       img.list = img_list)
}
```



The source code of the function is also available in the Data processing functions section of the data.helper.functions.R script on *GitHub*.

5.1.1.2 img.file_path.get_list Function

This function traverses *image directories*, collects *file paths* grouped by *folder* (representing *class labels*), and optionally limits, seeds, or randomly samples files for *dataset preparation*.

5.1.1.2.1 Usage



In the *code snippet* above, *this function* is called internally from the outer_img.file_path.get_list function described above in *this Section*.
In the *code snippet* above, a pointer to *this function* is passed in the fn.load_image.dat argument to the load_image.list function described above in *this Section*.

5.1.1.2.2 Arguments

- **root_path:** The root directory path containing subfolders of images organized by character recognition classes;
- **folder.list:** (Optional, *Character*, $NULL$ by default) Vector of specific subfolder names to process. If $NULL$, it defaults to all directories found inside the *root path* passed in the root_path argument;
- **char_files.max:** (Optional, *Integer*, NA by default) Numeric value limiting the maximum number of files to list per folder (useful for subsampling or testing);
- **char_files.seed:** (Optional, *Integer*, NA by default) Random seed used for reproducible sampling when char_files.max is specified (ignored otherwise);
- **random_sample:** (Optional, *Boolean*, $FALSE$ by default) Boolean ($TRUE/FALSE$) indicating whether files within a folder should be sampled randomly ($TRUE$) or taken sequentially from the top ($FALSE$).

5.1.1.2.3 Details

Below is the *step-by-step execution flow* of the function:

1. **Subfolder Discovery:** If the folder.list argument value is *NULL* (the *folder list* is not provided), the function retrieves all subdirectory names inside the *root directory* (using the path passed in the root_path argument) and uses them as the *folder list* for further processing. Otherwise, the folder.list value is used.



This is an auxiliary function that is used internally in the function as shown above in the Usage section of *this Description*.

5.1.1.2.4 Value

A named nested list where each element represents a subfolder, containing its root path and the file path list.

5.1.1.2.5 Source Code

The source code of the function is shown below:

```
img.file_path.get_list <- function(root_path,
                                   folder.list = NULL,
                                   char_files.max = NA,
                                   seed = NA,
                                   random_sample = FALSE) {
  if(is.null(folder.list))
    # Scans the root path to find all available subfolders.
    folder.list <- dir(root_path)

  # Generates a file path list
  path_list <- lapply(folder.list, function(folder_name){
    # Constructs the full path and retrieves all file paths for the given folder name.
    file.root_path <- file.path(root_path, folder_name)
    folder.idx <- which(folder.list == folder_name)

    put_log("Function: `img.file_path.get_list`:
Getting file path list from the following char's root folders:
%1", file.root_path)
```

```
    fpath.list <- list.files(file.root_path,
                             full.names = TRUE)

    fpath.len <- length(fpath.list)

    # If the number of files is limited and
    # the total number of files in the folder exceeds the limit
    if(!is.na(char_files.max) && fpath.len > char_files.max){

      # If a seed is provided.
      if (!is.na(seed)) {
        # Applies a reproducibility seed offset by the folder's index
        set.seed(seed + folder.idx)
      }


      if(random_sample) {
        # Samples the files randomly
        file.idx <- sample(fpath.len, size = char_files.max)
      } else {
        # Selects the top file according to the specified limit.
        file.idx <- seq_len(char_files.max)
      }

      fpath.list <- fpath.list[file.idx]
    }

    put_log("Function: `img.file_path.get_list`:
%1 files in folder: %2", length(fpath.list), folder_name)

    list(root_path = file.root_path,
         file_path.list = fpath.list)
  })

  # Shortens the name of each list item to the first character of the folder name..
  names(path_list) <- folder.list |> substr(1,1)
  path_list
}
```

5.1.1.3 load.img_trimmed.dat Function

This specialized utility function reads an image file, dynamically detects whether it contains nonzero pixel data, optionally trims whitespace or empty borders around the visual features using the magick package, and extracts its raw data matrix structure.

5.1.1.3.1 Usage

```
train.img_dat.list <- load_image.list(root_path = img.train_root.dir,
                                      fn.load_image.dat = load.img_trimmed.dat,
                                      folder.list = NULL,
                                      char_files.max = NA,
                                      char_files.seed = NA,
                                      random_sample = FALSE)
```



In the *code snippet* above, a pointer to *this function* is passed in the fn.load_image.dat argument to the load_image.list function described above in *this Section*.

5.1.1.3.2 Arguments

- **file_path:** Path to the image file to load.

5.1.1.3.3 Details

Below is the *step-by-step execution flow* of the function:

1. ***Image Loading:*** The function begins by reading the image file from disk into memory as an object using magick::image_read.
2. ***Matrix Conversion & Validation:*** Converts the loaded image into a numeric matrix representation using the user-defined helper function magick_img2matrix described below in described below in the Image Processing section.

3. ***Image Validation & Conditional Cropping:*** Evaluates the matrix's maximum value to check whether the image is entirely blank or empty. If valid content exists (the maximum pixel value is greater than zero), it applies magick::image_trim to automatically crop out uniform background borders or whitespace, centering the content around the core visual features.

4. ***Data Extraction:*** Finally, it extracts and returns the underlying raw pixel buffer data using the magick::image_data helper function.



This is an auxiliary function, the pointer to which is passed in the fn.load_image.dat argument to the load_image.list function, as shown above in the Usage section of *this Description*.

5.1.1.3.4 Value

5.1.1.3.5 Source Code

The source code of the function is shown below:

```
load.img_trimmed.dat <- function(file_path) {
  img <- magick::image_read(file_path)
  img.mx <- magick_img2matrix(img)
  img.trimmed <- img

  if (max(img.mx) > 0) {
    img.trimmed <- image_trim(img)
  }

  magick::image_data(img.trimmed)
}
```

5.1.2 Image Processing {#appndx_a@img_proc}



The complete source code of the functions described in this section is available in the Image Processing section of the data-helper.R script on *GitHub*.

5.1.2.1 magick_img2matrix Function

This is a lightweight utility that converts a Magick Image object into a *normalized numeric matrix of pixel intensities*.

5.1.2.1.1 Usage

```
img.mx <- magick_img2matrix(img)
```



In the *code snippet* above, *this function* is called internally from the load.img_trimmed.dat function described above in *this Section*.

5.1.2.1.2 Arguments

- **img:** A Magick Image object (typically processed, resized, or filtered prior to conversion).

5.1.2.1.3 Details

Below is the *step-by-step execution flow* of the function:

1. (Line 482) ***Raw Binary Data Extraction:*** The function calls the [image_data] function to extract the underlying raw hexadecimal or integer byte-array buffer from the Magick Image object.
2. (Line 483) ***Color Channel Isolation:*** Slices the multidimensional array to isolate the primary color channel matrix. For grayscale or black-and-white pipeline images (as in our case), this captures the single intensity plane.

3. (Line 484) ***Data Type Conversion:*** Explicitly casts the extracted data array elements from raw/character bytes into a numeric data type, making mathematical operations possible.
4. (Line 485) ***Min-Max Normalization:*** Divides the entire matrix by 255 (the maximum 8-bit color intensity value) to normalize pixel values into a clean floating-point scale ranging from 0.0 (black/background) to 1.0



This is a utility function used internally by the other user-defined functions, as shown in the usage example above in the Usage section of *this Description*.

5.1.2.1.4 Value

5.1.2.1.5 Source Code

The source code of the function is shown below:

```
magick_img2matrix <- function(img){
  img.dat <- image_data(img)
  mx.bin <- img.dat[1,,]
  mode(mx.bin) <- "numeric"
  mx.bin /255
}
```

Chapter 6

Chapter 7

Bibliography

[1] Rafael A. Irizarry. *Introduction to Data Science, Part I. Data Wrangling and Visualization with R*. Oct. 24, 2019. URL: https://rafalab.dfci.harvard.edu/dsbook-part-1/ (cit. on p. 4).

[2] Rafael A. Irizarry. *Introduction to Data Science, Part II. Statistics and Prediction Algorithms Through Case Studies*. Dec. 27, 2024. URL: https://rafalab.dfci.harvard.edu/dsbook-part-2/ (visited on 02/18/2025) (cit. on p. 4).

[3] Ajay Agarwal, Sartaj Ahmad, and Minakshi. "Handwritten Character Recognition: A Comprehensive Survey". In: *2023 Second International Conference on Informatics (ICI)*. 2023, pp. 1–6. DOI: 10.1109/ICI60088.2023.10420981. URL: https://ieeexplore.ieee.org/document/10420981 (cit. on p. 5).

[4] Azamat Kurbanaev. *edX Data Science: Capstone, Choose Your Own. Handwritten Character Recognition Multiclass Classifier*. Version 1.0.0. Oct. 5, 2026. URL: https://github.com/AzKurban-edX-DS/edx.capstone.choose-your-own (cit. on p. 5).

[5] Tahir Cetin Akinci, Oguzhan Topsakal, and M. Ilhan Akbas. "Machine Learning Methods from Shallow Learning to Deep Learning". In: *Shallow Learning vs. Deep Learning: A Practical Guide for Machine Learning Solutions*. Ed. by Ömer Faruk Ertuğrul, Josep M. Guerrero, and Musa Yilmaz. Cham: Springer Nature Switzerland, 2024, pp. 1–28. ISBN: 978-3-031-69499-8. DOI: 10.1007/978-3-031-69499-8_1. URL: https://doi.org/10.1007/978-3-031-69499-8_1 (cit. on p. 6).